COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CpE 010 - Data Structures and Algorithms	**SECTION:**	CpE21S1
DATE PERFORMED:	9-8-2026	**DATE SUBMITTED:**	9-9-2026
NAME:	Mauro, Ryan Lewis B.	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7.1 [Sorting Algorithms]

1. PROCEDURE OUTPUT

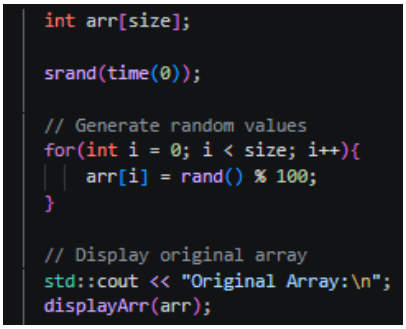
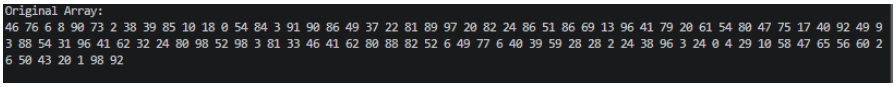
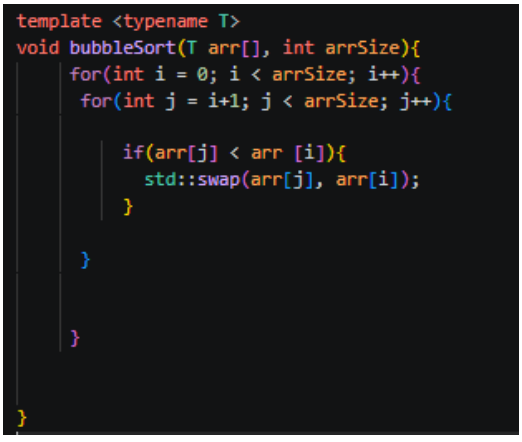
Code	
Console Screenshot	
Observations	The program successfully generated 100 random integer values ranging from 0 to 99. The values are initially unsorted and contain some duplicate numbers. This original array is used as the input for testing the different sorting algorithms.

Table 7-1. Array of Values for Sort Algorithm Testing

Code	

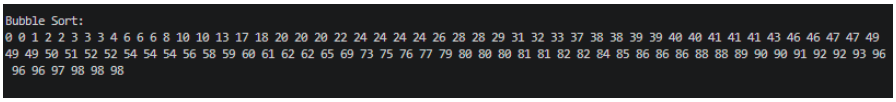
Console Screenshot	
Observations	Bubble Sort successfully arranged all 100 values in ascending order. The algorithm compares the elements and swaps them when they are in the wrong order. The result shows that the smallest values are placed at the beginning while the larger values move toward the end.

Table 7-2. Bubble Sort Technique

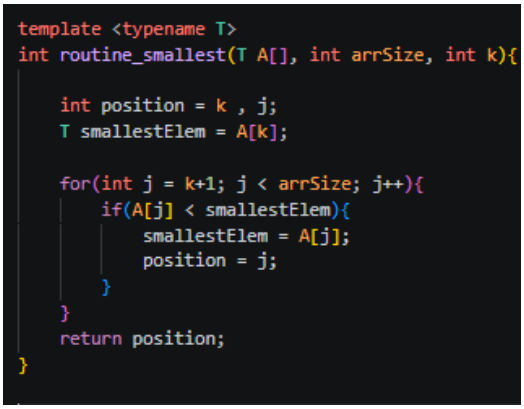
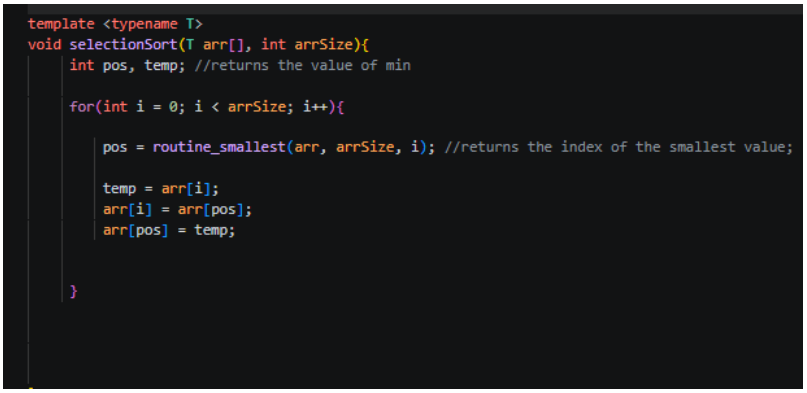
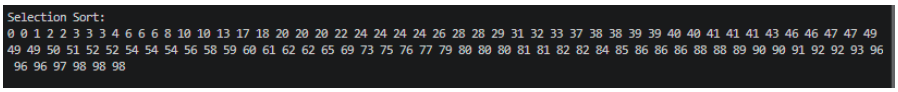
Code	 
Console Screenshot	
Observations	Selection Sort successfully sorted the 100 values in ascending order. It works by searching for the smallest value in the unsorted part of the array and placing it in the correct position. The duplicate values remained in the array after sorting.

Table 7-3. Selection Sort Algorithm

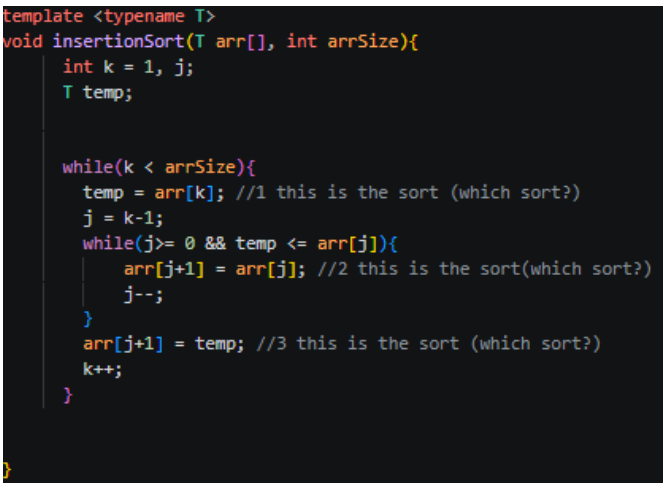
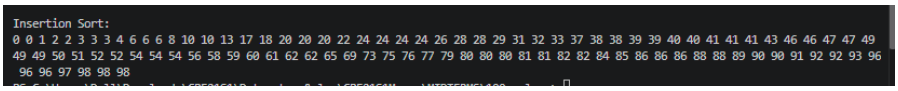
Code	
Console Screenshot	
Observations	Insertion Sort successfully arranged the 100 values in ascending order. It works by taking one element at a time and inserting it into its proper position among the previously sorted elements. The final output contains the same values as the original array but arranged from smallest to largest.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

START

Create an array A with 101 elements

Fill the array with random values from 1 to 5

```
FOR i = 0 to 100
   FOR j = 0 to 99 - i
      IF A[j] > A[j + 1]
         Swap A[j] and A[j + 1]
      END IF
   END FOR
END FOR
```

Set count1 = 0
Set count2 = 0
Set count3 = 0
Set count4 = 0

```
Set count5 = 0

FOR i = 0 to 100
   IF A[i] = 1
      count1 = count1 + 1
   ELSE IF A[i] = 2
      count2 = count2 + 1
   ELSE IF A[i] = 3
      count3 = count3 + 1
   ELSE IF A[i] = 4
      count4 = count4 + 1
   ELSE IF A[i] = 5
      count5 = count5 + 1
   END IF
END FOR

Display the sorted array
Display the vote count for each candidate

END
```

Table 7-5. Pseudocode of Algorithm

The pseudocode describes the step-by-step process of the vote-counting algorithm before writing the actual program. First, the program creates an array containing 101 votes with values from 1 to 5, where each number represents a candidate. The program then sorts the votes using Bubble Sort in ascending order. After sorting, the program counts how many times each candidate number appears in the array. Finally, it displays the total number of votes received by each candidate. The pseudocode helps organize the logic of the program and makes the actual coding process easier to understand.

<table>
<tr>
<td>

```
template <typename T>
void bubbleSort(T arr[], int arrSize){

    for(int i = 0; i < arrSize; i++){

        for(int j = 0; j < arrSize - 1; j++){

            if(arr[j] > arr[j + 1]){
                std::swap(arr[j], arr[j + 1]);
            }

        }

    }

}
```

</td>
<td>The bubbleSort() function sorts the elements of the array in ascending order. It uses two loops to compare the elements in the array. If the element at position j is smaller than the element at position i, the two elements are swapped. This process continues until all the values are arranged from the smallest to the largest.

Why did I choose this Sorting Algorithm?
I chose Bubble Sort because it is simple and easy to understand and implement. It is suitable for this activity because the program only contains 101 vote values. Bubble Sort can arrange the votes from the smallest value to the largest value, making it easier to group and count the votes for each candidate. Although other sorting algorithms such as Selection Sort and Insertion Sort could also be used, I chose Bubble Sort because its comparison and swapping process is straightforward and easy to demonstrate.</td>
</tr>
</table>

Table 7-6. Screenshot of Algorithm Code/Justification

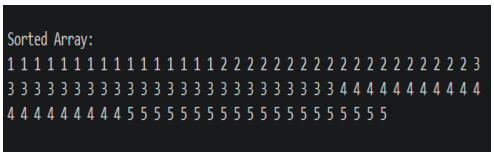
Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
	Candidate 1 - Bo Dalton Capistrano: **16 votes**	Candidate 1 - Bo Dalton Capistrano: **16 votes**
	Candidate 2 - Cornelius Raymon Agustin: **19 votes**	Candidate 2 - Cornelius Raymon Agustin: **19 votes**
	Candidate 3 - Deja Jayla Banaga: **26 votes**	Candidate 3 - Deja Jayla Banaga: **26 votes**
	Candidate 4 - Laila Brielle Yabut: **20 votes**	Candidate 4 - Laila Brielle Yabut: **20 votes**
	Candidate 5 - Franklin Relano Castro: **20 votes**	Candidate 5 - Franklin Relano Castro: **20 votes**

Table 7-7. Output Testing

During the output testing, the program successfully generated 101 votes and arranged them from the smallest value to the largest using Bubble Sort. The sorted array made it easier to identify and count the votes for each candidate. The algorithm produced the following results: Candidate 1 received 16 votes, Candidate 2 received 19 votes, Candidate 3 received 26 votes, Candidate 4 received 20 votes, and Candidate 5 received 20 votes. The total is 101 votes, which matches the required number of votes. The manual count also matched the algorithm's results, showing that the program worked correctly.

Question: Was your developed vote counting algorithm effective? Why or why not?

Yes, the developed vote-counting algorithm was effective. It successfully sorted the 101 votes and correctly counted the votes for all five candidates. The results produced by the algorithm matched the manual count, with a total of 101 votes. This shows that the algorithm was able to perform its intended task accurately and consistently.

Conclusion/Learnings

I think I did well in this activity because I was able to develop a working vote-counting program using Bubble Sort and apply it to a real example. I learned how sorting algorithms can make data easier to organize and count, and I also learned the importance of testing the program and comparing the results with a manual count. However, I still need to improve my understanding of sorting algorithms, especially how the loops and comparisons work. I also need to improve my coding skills so I can create more efficient and organized programs in the future.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.